Name: Cohort/Batch: Date: Score:

HIBERNATE PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 Q&A on ORM, caching, N+1, lazy loading, and migrations

TOTAL: 10 QUESTIONS

Q1 What is Hibernate and how does it map Java objects to database tables? Threat Model

Q2 Explain the Hibernate entity lifecycle states. Architecture

Q3 What is the difference between Session.get() and Session.load()? Configuration

Q4 What is lazy loading and when does it cause issues? Hardening

Q5 Describe Hibernate's two levels of caching. SSO / IdP

Q6 How do you map a bidirectional @OneToMany relationship? Observability

Q7 What is the N+1 select problem and how do you fix it? Lifecycle

Q8 How does optimistic locking work in Hibernate with @Version? Compliance

Q9 What is HQL and how does it differ from SQL and Criteria API? Tuning

Q10 How do you manage schema migrations with Flyway and Hibernate? Tuning

ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 **Hibernate is a JPA-compliant ORM. @Entity marks** Mitigation

Hibernate is a JPA-compliant ORM. @Entity marks a class as a persistent entity. Field annotations (@Column, @Id) map attributes to columns. At startup Hibernate builds a Metamodel and can auto-generate DDL from the mappings.

A2 **Transient: new object, unknown to Hibernate. Persistent:** Primitives

Transient: new object, unknown to Hibernate. Persistent: associated with a Session; changes are tracked. Detached: Session closed, changes not tracked. Removed: scheduled for DELETE. Calling session.save() moves transient to persistent.

A3 **get() hits the database immediately and returns** Hardening

get() hits the database immediately and returns null if not found. load() returns a proxy (lazy placeholder) and defers the SQL hit until a field is accessed. load() throws ObjectNotFoundException at access time if the row is missing.

A4 **Lazy loading defers the SQL for associations** Gotchas

Lazy loading defers the SQL for associations (e.g., @OneToMany) until the collection is accessed. It causes LazyInitializationException when accessed outside an open Session. Fix with JOIN FETCH, @EntityGraph, or keeping the Session open (Open Session in View).

A5 **L1 (Session cache) is always on; identical** Federation

L1 (Session cache) is always on; identical IDs within one Session return the same object without a second SQL query. L2 (SessionFactory cache, optional) is shared across Sessions; providers include Ehcache and Redis. @Cacheable enables it per entity.

A6 **The @OneToMany side uses mappedBy='fieldName' pointing** Observability

The @OneToMany side uses mappedBy='fieldName' pointing to the owning side. The @ManyToOne side owns the foreign key column. Always update the owning side to persist changes; the inverse side is for navigation only.

A7 **N+1 occurs when loading N parent entities** Automation

N+1 occurs when loading N parent entities then running one query per entity to fetch a child collection. Fix with JOIN FETCH in JPQL (loads all in one query), @EntityGraph, or batch-fetching via @BatchSize on the association.

A8 **Add a @Version Long field to the** Governance

Add a @Version Long field to the entity. Hibernate includes WHERE version=? in every UPDATE. If another transaction modified the row first, the version no longer matches and Hibernate throws OptimisticLockException, preventing a lost update.

A9 **HQL is object-oriented: queries reference entity names** Assessment

HQL is object-oriented: queries reference entity names and field names, not table or column names. Hibernate translates HQL to the target SQL dialect. The Criteria API constructs the same queries programmatically in a type-safe manner.

A10 **Set spring.jpa.hibernate.ddl-auto=validate so Hibernate only** Optimization

Set spring.jpa.hibernate.ddl-auto=validate so Hibernate only checks schema matches entities without changing it. Flyway applies versioned SQL scripts (V1__init.sql, V2__add_column.sql) in order, keeping schema changes explicit and auditable.